Predicting NBA Field Goals: A Bayesian Logistic Model Estimated by MCMC

Group 28 (ICA2)

Contents

Introduction

In basketball analytics, a fundamental question is whether any given shot will be made or missed. This is a binary outcome of interest to coaches, players, and analysts, whose probability is worth modelling both for prediction ("how likely is this shot to go in?") and interpretation ("which factors make a shot more difficult?"). Logistic regression is a natural baseline for this task because it links the "make" probability to contextual covariates such as shot distance, defender proximity, time remaining, and other shot-creation indicators, to finally summarise each effect through interpretable odds ratios. In the classical frequentist framework, parameters are typically estimated by maximum likelihood, solved through iterative optimisation routines, and uncertainty statements are usually justified using large-sample results. However, these standard procedures can perform poorly in practice, as optimisation may struggle to converge under challenging data configurations, and maximum-likelihood estimates can be noticeably biased when sample sizes are small. A Bayesian formulation of logistic regression provides an alternative that can solve these issues. By treating parameters as random variables and combining prior information with the likelihood, Bayesian estimation offers greater flexibility and is less dependent on the strong regularity conditions that underlie classical inference. This approach is often implemented using Markov Chain Monte Carlo (MCMC) sampling, specifically a Metropolis–Hastings algorithm, which has matured to the point of supporting a wide range of nonlinear regression models. Despite these methodological advances, Bayesian logistic regression remains underused in many applied settings, partly because the underlying concepts and practical implementation details are less familiar. While Bayesian logistic models have been adopted in several disciplines, there is still limited work within sports analytics.

Literature Review

Several studies and exercises model field-goal outcomes using classical (MLE) logistic regression to model the binary result "made" vs "miss". For example, Cheema et al. "Modelling Shot Probability in the NBA" models NBA field-goal success as a binary response using logistic regression to estimate shot difficulty from contextual predictors. It discusses covariate selection, expresses multicollinearity concerns as well as limitations of the data set. Furthermore, in a later study Terhanian's "Advice on Making the Most of Basketball Three-point Shot Data" analyses tens of thousands of the three-point attempts from the same dataset once more using logistic regression to report that variables such as defender distance, shot clock time, catch and dribble as well as shot distance all significantly change "make" and "miss" outcome probabilities. The study proposes several simulation tools to translate alterations in the above factors into expected changes in three-point percentages of scoring (Terhanian (2015)). Edmond's How can an NBA Player be Clutch (Edmond (2017)) also used a logistic regression classification model to find significance in covariates in baskets being made under a specific set of parameters. Defining "clutch" as: situations in which a shot was taken in the final two minutes of regulation or overtime in a game decided by 5 or fewer points. Using once again the same dataset, that was the last time the NBA provided free detailed player movement and scoring data. The paper analyses separate findings for clutch vs non-clutch situations and uses odds ratios to describe the outcomes. Its analysis of the data statistically proves situations that seem reasonable for someone following the sport, such as increased ball-holding and 3-point selection during "clutch time". The above papers collectively show that classical logistic regression evidently provides an accurate baseline for estimating how shot-context variables can affect the "make" and "miss" probabilities. Bayesian logistic regression in other fields is often used to compare to the classical. Some papers of reference include Mila Michailides' paper on plant pathology (Mila and Michailides (2008)). The results found using Bayesian methods to disease prediction for pistachio blight improved significantly predictive performance when used to update parameter estimates with new-season information introduced relative to the frequentist analysis. In clinical modelling, Gordovil-Merino et al. compared the two methods of estimation on some medical data of ADHD cases (n=286) to find that the Bayesian model substantially stabilised inference through prior information. More specifically, the use of the Bayesian framework substantially reduced coefficient standard errors, yielding more stable estimates. As described in their study, this type of analysis can be particularly useful for complex models where parameter estimation may be easily compromised (Gordovil-Merino et al. (2010)).

Some Bayesian approaches exist on NBA shot-make probabilities, such as Cao et al.'s "What influences the Field Goal Attempts of Professional Players? Analysis of Basketball Shot Charts via Log Gaussian Cox Process with Spatially Varying Coefficients" (Cao et al. (2025)). However, the Bayesian logistic regression is less commonly used for these models, especially with a transparent MCMC implementation. Therefore, our aim is to apply the Bayesian logistic regression to model NBA shot outcomes, implementing a Metropolis Hastings algorithm from scratch to sample from the posterior and obtain predicted "make" probabilities. We will then use K-fold cross validation to evaluate whether this method provides better estimates compared to the basic logistic regression.

1. Methodology and Theoretical Framework

1.1 The Bayesian Approach

The main distinction between Bayesian and frequentist statistics is how they treat unknown parameters. In the frequentist framework, model parameters are fixed but unknown values and the data are considered random. In Bayesian statistics the randomness comes from the parameters themselves and they are treated as random variables with prior distributions that are updated using the observed data.

Consider a model with outcome y and unknown parameter vector θ.

Bayesian inference starts by specifying:

- A prior distribution $p(\theta)$, describing plausible values for θ before observing the data.
- A likelihood function $p(y \mid \theta)$, which describes how the data is generated, given θ.

After observing the data, we update our beliefs using Bayes' theorem to find the posterior distribution:

$$p(\theta \mid y) = \frac{p(y \mid \theta)\, p(\theta)}{p(y)}, \qquad p(y) = \int p(y \mid \theta) p(\theta)\, d\theta$$

Where $p(y)$ is the marginal likelihood, which can be rewritten using the law of total probability (see above). It is important to note that $p(y)$ is simply a constant and it ensures that $p(\theta \mid y)$ integrates to 1.

In many cases $p(y)$ involves integrals that are too complicated or impossible to solve analytically, therefore when finding a posterior distribution we often only look at the posterior up to proportionality:

$$p(\theta \mid y) \propto p(y \mid \theta) p(\theta),$$

Using this, we may be able to identify the general form of the posterior distribution (e.g. normal, gamma, exponential…) and then derive the normalizing constant. However in many cases it is not possible to identify the distribution, so it is not possible to find a closed form for the posterior distribution. In these cases we rely on simulations, such as MCMC, to sample from the posterior distribution.

Once the posterior distribution has been obtained (or simulated), we can create estimates for θ and quantify uncertainty.

The most common point estimates for θ are:

- The posterior mean: $\mathbb{E}[\theta \mid y]$
- The posterior median: $\text{Med}(\theta \mid y)$

To quantify the uncertainty around θ we may use the posterior variance $\text{Var}(\theta \mid y)$. It is also possible to create credible intervals using posterior quantiles, which are similar to confidence intervals in frequentist inference. For example a 95% credible interval is given by: $\left[q_{0.025}(\theta_j \mid y),\ q_{0.975}(\theta_j \mid y)\right]$.

1.2 The Bayesian Logistic Regression

1.2.1 Our Data

For this project, our response variable is whether or not a shot was made, which is a binary response variable. For each shot i define:

$$y_i = \begin{cases} 1, & \text{if the shot is made} \\ 0, & \text{if the shot is missed.} \end{cases}$$

Let $x_i = (x_{i1}, \dots, x_{ip})^\top$ denote the vector of covariates/predictors associated with shot i (e.g. shot time, distance, location, etc.). Our objective is to model the probability of a specific shot being made given these covariates.

1.2.2 Building the Classic Logistic Regression

The logistic regression allows us to model the effect of a set of covariates (x_i) on the probability of a binary outcome y_i.

The quantity of interest is therefore:

$$\mathbb{P}(y_i = 1 \mid x_i) \equiv p_i$$

A first idea might be to model the probabilities a linear function of the covariates, i.e.:

$$p_i = x_i^\top \beta$$

However, this would not be suitable, as a linear combination $x_i^\top \beta$ can take any value in $\mathbb{R}$ and we need $p_i \in [0, 1]$. Therefore we need a model that maps the predictor $x_i^\top \beta$ into a valid probability. To overcome this problem, the logistic regression does not model p_i itself, rather a transformation of p_i that can take any real value.

First, consider the odds of success. For shot i the probability of success is $p_i = \mathbb{P}(y_i = 1 \mid x_i)$, so the probability of failure is $1 - p_i$. Therefore we can define the odds of success as:

$$\frac{p_i}{1 - p_i}$$

For any valid probability (i.e. $p_i \in [0, 1]$) the odds of success will be in the interval $(0, \infty)$, which is still not ideal for linear modelling. To remove this restriction we may take the logarithm of the odds, i.e. the log-odds or logit:

$$\log\left(\frac{p_i}{1 - p_i}\right)$$

This function (log-odds) is defined in the interval $(-\infty, \infty)$ and only for $p_i \in [0, 1]$, which is exactly what we need. As a result, we may model the transformed quantity as a linear combination of the predictors/covariates.

A Logistic regression therefore assumes that:

$$\log\left(\frac{p_i}{1 - p_i}\right) = x_i^\top \beta$$

Now we can now solve for p_i:

$$\left(\frac{p_i}{1-p_i}\right) = e^{x_i^\top \beta}$$
$$p_i = e^{x_i^\top \beta} \cdot (1-p_i)$$
$$p_i = e^{x_i^\top \beta} - p_i \cdot e^{x_i^\top \beta}$$
$$p_i + p_i \cdot e^{x_i^\top \beta} = e^{x_i^\top \beta}$$
$$p_i \cdot (1 + e^{x_i^\top \beta}) = e^{x_i^\top \beta}$$
$$p_i = \frac{e^{x_i^\top \beta}}{1 + e^{x_i^\top \beta}} = \frac{1}{1 + e^{-x_i^\top \beta}}$$

This is the logistic function that maps any number in $\mathbb{R}$ into a probability $p_i \in (0,1)$. We can therefore write the outcome y_i conditioned on the vector of coefficients and covariates as a Bernoulli distribution with success probability p_i, i.e.:

$$y_i \mid x_i, \beta \sim \text{Bernoulli}(p_i), \quad p_i = \frac{1}{1 + e^{-x_i^\top \beta}}$$

The coefficient vector $\beta = (\beta_0, \beta_1, \ \dots \ , \beta_p)^\top$ determines how the covariates affect the probability of making a shot (success), since:

$$\log\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \beta_1 \cdot x_{i1} + \ \dots \ + \beta_p \cdot x_{ip}$$

Each coefficient β_j represents the change in the log-odds of making a shot after a one unit increase in x_{ij}.

Therefore:

- if $\beta_j > 0$, increasing x_{ij} increases the probability of success.
- if $\beta_j < 0$, increasing x_{ij} decreases the probability of success.
- if $\beta_j = 0$, x_{ij} has no effect on the probability of success.

1.2.3 The Likelihood Function

We have now derived an expression for the probability of making a shot in terms of the covariates and their respective coefficients. Since the outcome y_i conditioned on the vector of coefficients and covariates follows a Bernoulli distribution, we can write the probability mass function:

$$\mathbb{P}(y_i \mid x_i, \beta) = p_i^{y_i} \cdot (1-p_i)^{1-y_i}$$

Therefore, if we assume the observations are conditionally independent given β, we can write the likelihood of the whole sample as:

$$\mathcal{L}(\beta) = p(y \mid X, \beta) = \prod_{i=1}^{n} p_i^{y_i} \cdot (1-p_i)^{1-y_i}$$

In the classical logistic regression, estimates are found by maximizing the likelihood with respect to β (Maximum Likelihood).

1.2.4 Moving from Frequentist to Bayesian

Now we will consider the Bayesian logistic regression, which is the model we will be using. The classical/frequentist and Bayesian versions use the same likelihood, the key difference is how they treat the unknown parameters β.

In the classical framework we considered β as a fixed but unknown quantity, but now we will regard β as a random variable with a probability distribution that we aim to find.

We start by specifying a prior distribution $p(\beta)$ before we observe the data. This will reflect any previous knowledge we have of the data and how we expect it to behave.

After we observe the data we update our knowledge of the parameter β and find a posterior distribution using Bayes' theorem:

$$p(\beta \mid y, X) \propto p(y \mid X, \beta) \cdot p(\beta)$$

Substituting the likelihood for the logistic regression gives:

$$p(\beta \mid y, X) \propto \left(\prod_{i=1}^{n} p_i^{y_i} \cdot (1 - p_i)^{1-y_i} \right) \cdot p(\beta), \quad p_i = \frac{1}{1 + e^{-x_i^\top \beta}}$$

In the logistic regression, the likelihood involves a non-linear logistic function, so the posterior does not belong to a standard family of distributions, which makes it impossible to identify the form of the distribution. In addition, the normalizing constant

$$p(y) = \int p(y \mid X, \beta) p(\beta) \, d\beta$$

cannot be computed in closed form.

For this reason, finding the exact form of the distribution can be difficult or impossible. Instead, we may use MCMC to simulate the posterior and obtain the desired estimates. In particular, we will be using the Metropolis Hastings algorithm.

1.3 The Metropolis Hastings Algorithm

1.3.1 Procedure

In Bayesian logistic regression, our distribution of interest is the posterior distribution of the coefficient vector β. As seen before, in this case, we can only write this posterior up to proportionality. We can call this the target unnormalized density, which we will denote as $\tilde{\pi}(\beta)$ for simplicity:

$$p(\beta \mid y, X) \propto p(y \mid X, \beta) \cdot p(\beta) := \tilde{\pi}(\beta)$$

Therefore the posterior may be written as:

$$p(\beta \mid y, X) = \frac{p(y \mid X, \beta) \cdot p(\beta)}{C} = \frac{\tilde{\pi}(\beta)}{C}$$

Where C is the unknown normalizing constant. This will be our target distribution.

The Metropolis Hastings algorithm constructs a Markov Chain which has a stationary distribution equal to our target distribution. The algorithm starts by specifying a starting value $\beta^{(0)}$ and a proposal distribution $Q\left(\beta' \mid \beta^{(t)}\right)$. Given a current value $\beta^{(t)}$, a new value β' is generated from the proposal distribution. This

value is then accepted or rejected according to an acceptance probability. The algorithm keeps running until the chain has converged to the target distribution.

More specifically, the algorithm runs as follows. Assume we are currently at iteration t, first we generate a value:

$$\beta' \sim Q\left(\ \cdot \mid \beta^{(t)}\right)$$

Next compute the acceptance probability:

$$\alpha\left(\beta' \mid \beta^{(t)}\right) = \min\left\{\frac{\tilde{\pi}\left(\beta'\right)}{\tilde{\pi}\left(\beta^{(t)}\right)} \cdot \frac{Q\left(\beta' \mid \beta^{(t)}\right)}{Q\left(\beta^{(t)} \mid \beta'\right)},\ 1\right\}$$

The proposed value β' will then be accepted with probability $\alpha\left(\beta' \mid \beta^{(t)}\right)$. If the value is accepted, then:

$$\beta^{(t+1)} = \beta'$$

Otherwise, if the value is rejected, the chain remains in its current value:

$$\beta^{(t+1)} = \beta^{(t)}$$

Repeating this procedure, generates the sequence:

$$\beta^{(0)},\ \beta^{(1)},\ \beta^{(2)},\ ...$$

which after a sufficient amount of iterations provides sample draws from the target distribution.

1.3.2 The Acceptance Probability

The acceptance probability is the key feature of this algorithm and it ensures that the chain converges into the desired stationary distribution $\pi(\beta) = p(\beta\ \mid y, X)$.

Suppose the chain is currently in state β and β' is proposed. The transition density for moving from β to β' is therefore:

$$P(\beta' \mid \beta) = Q(\beta' \mid \beta) \cdot \alpha(\beta' \mid \beta), \quad \text{for} \quad \beta' \neq \beta$$

For π to be a stationary distribution, the following condition needs to hold:

$$\pi(\beta) \cdot P(\beta' \mid \beta) = \pi(\beta') \cdot P(\beta \mid \beta')$$

This is the detailed balance condition and it means that, under the target distribution π, the density flowing from state β to β' is exactly the same as the density flowing from β' to β, i.e. there is no net gain or loss at either point and π is a stationary distribution.

We can show that this condition holds for the acceptance probability we defined earlier. Start with the left hand side of the condition:

$$\pi(\beta) \cdot Q(\beta' \mid \beta) \cdot \alpha(\beta' \mid \beta)$$

Substitute the expression for the acceptance probability:

$$\pi(\beta) \cdot Q(\beta' \mid \beta) \cdot \min\left\{1, \frac{\tilde{\pi}(\beta') \cdot Q(\beta \mid \beta')}{\tilde{\pi}(\beta) \cdot Q(\beta' \mid \beta)}\right\}$$

Rewrite $\pi(\beta)$ as $\frac{\tilde{\pi}(\beta)}{C}$ and combine terms:

$$\min\left\{\frac{\tilde{\pi}(\beta)}{C} \cdot Q(\beta' \mid \beta),\ \frac{\tilde{\pi}(\beta)}{C} \cdot Q(\beta' \mid \beta) \cdot \frac{\tilde{\pi}(\beta') \cdot Q(\beta \mid \beta')}{\tilde{\pi}(\beta) \cdot Q(\beta' \mid \beta)}\right\}$$

Simplify

$$\min\left\{\frac{\tilde{\pi}(\beta)}{C} \cdot Q(\beta' \mid \beta),\ \frac{\tilde{\pi}(\beta')}{C} \cdot Q(\beta \mid \beta')\right\}$$

Now replace $\tilde{\pi}(\beta)/C$ with $\pi(\beta)$, and similarly for β':

$$\min\left\{\pi(\beta) \cdot Q(\beta' \mid \beta),\ \pi(\beta') \cdot Q(\beta \mid \beta')\right\}$$

Now, we may use the same working out as above to get the following expression for the right hand side of the condition:

$$\pi(\beta') \cdot P(\beta \mid \beta') = \min\left\{\pi(\beta') \cdot Q(\beta \mid \beta'),\ \pi(\beta) \cdot Q(\beta' \mid \beta)\right\}$$

Since $\min\{A, B\} = \min\{B, A\}$:

$$\min\left\{\pi(\beta) \cdot Q(\beta' \mid \beta),\ \pi(\beta') \cdot Q(\beta \mid \beta')\right\} = \min\left\{\pi(\beta') \cdot Q(\beta \mid \beta'),\ \pi(\beta) \cdot Q(\beta' \mid \beta)\right\}$$

And therefore:

$$\pi(\beta) \cdot P(\beta' \mid \beta) = \pi(\beta') \cdot P(\beta \mid \beta')$$

As required. Hence, we have shown that the detailed balance condition holds, meaning that for the defined acceptance probability, the chain will converge to the target distribution π.

1.4 Testing and Performance: K-Fold Cross Validation

After the model has been specified, it is important that we assess how well it generalises beyond the data used for the estimation. To do this we will be using out-of-sample testing, specifically K-fold cross validation.

K-fold cross validation starts by randomly dividing the data into K subsets or "folds", where K can be chosen arbitrarily. For each fold, $k = 1, 2, \ldots, K$, the k^{th} fold is treated as a validation set and the $K-1$ remaining folds are used as a training set. Then the model is estimated using the training set and evaluated using the validation set. This procedure will give K estimates of the predictive performance of the model, which can then be averaged to obtain a general estimate for the predictive performance.

More specifically, let $D = \{(x_i, y_i)\} i = 1^n$ be the full dataset and let $D\text{train}^{(k)}$ and $D^{(k)}_{\text{valid}}$ denote the training and validation set in fold k. In each fold, our model is fitted using $D^{(k)}_{\text{train}}$ producing posterior draws $\beta^{(1)}, ..., \beta^{(S)}$ via MCMC. Then, for each observation in the validation set, the posterior predictive probability is computed by averaging over all posterior draws

$$\hat{p}(x) = \frac{1}{S}\sum_{s=1}^{S} \sigma(x^\top \beta^{(s)}), \qquad \sigma(z) = \frac{1}{1+e^{-z}}$$

These predicted probabilities are used to compute the accuracy of the model. We will classify a shot as made if $\hat{p}(x) > 0.5$ and missed otherwise, and we will report the fraction of shots for which this classification matches the outcome, i.e.:

$$\hat{y} = \begin{cases} 1, & \hat{p}(x) > 0.5, \\ 0, & \hat{p}(x) \leq 0.5, \end{cases} \qquad \text{Accuracy}^{(k)} = \frac{1}{|D^{(k)}_{\text{valid}}|} \sum_{(x_i, y_i) \in D^{(k)}_{\text{valid}}} \mathbf{1}\{\hat{y}_i = y_i\}$$

Finally, once we have the accuracy for each fold, we may average across all folds to obtain an estimate for the general accuracy of our model:

$$\text{General Accuracy} = \frac{1}{K} \sum_{k=1}^{K} \text{Accuracy}^{(k)}$$

1.5 Software and Implementation

We will combine Python and R in order to take advantage of the strengths of both languages within the same analysis. Python is particularly useful for the data cleaning, manipulation, and exploratory analysis, as libraries such as pandas and matplotlib make it easy to work with large datasets and make clear plots. On the other hand, we will use R for the statistical modelling stage (implementing the Bayesian logistic regression, running the Metropolis–Hastings algorithm, and handling the cross-validation). Using both languages therefore allows the analysis to be carried out more effectively, with each language being used where it is most convenient and appropriate.

Having outlined the theoretical foundations of the methodology, we now proceed with our application to the NBA shots.

2. Exploratory Data Analysis

During the 2014-15 season, the NBA fully implemented the SportVU player-tracking camera technology in all 30 arenas. This camera allowed for further research on the games statistics, and provided a different point of view for the spectators. These data were published early on in the process of implementation, however, it became private in the following years, once they saw the real value of these for modelling odds and its commercial potential. We will use the dataset published in 2014-15 for the purpose of the announced model building. The dataset contains 128,069 observations (shot attempts) and 21 variables. Alongside game and player identifiers, it includes contextual shot information (period, game/shot clock, shot distance, closest-defender distance, etc.) and outcome variables such as Field Goal Made (`FGM`) and points (`PTS`). We now begin with the Exploratory Data Analysis.

We now look at the columns and first few rows to get a general idea of the data we are working with

```
df1.columns
```

```
## Index(['GAME_ID', 'MATCHUP', 'LOCATION', 'W', 'FINAL_MARGIN', 'SHOT_NUMBER',
##        'PERIOD', 'GAME_CLOCK', 'SHOT_CLOCK', 'DRIBBLES', 'TOUCH_TIME',
##        'SHOT_DIST', 'PTS_TYPE', 'SHOT_RESULT', 'CLOSEST_DEFENDER',
##        'CLOSEST_DEFENDER_PLAYER_ID', 'CLOSE_DEF_DIST', 'FGM', 'PTS',
##        'player_name', 'player_id'],
##       dtype='str')
```

```
df1.head()
```

```
##     GAME_ID                    MATCHUP LOCATION  ... PTS    player_name  player_id
## 0  21400899  MAR 04, 2015 - CHA @ BKN        A  ...   2  brian roberts     203148
## 1  21400899  MAR 04, 2015 - CHA @ BKN        A  ...   0  brian roberts     203148
## 2  21400899  MAR 04, 2015 - CHA @ BKN        A  ...   0  brian roberts     203148
## 3  21400899  MAR 04, 2015 - CHA @ BKN        A  ...   0  brian roberts     203148
## 4  21400899  MAR 04, 2015 - CHA @ BKN        A  ...   0  brian roberts     203148
##
## [5 rows x 21 columns]
```

Now, we check to see if there are any duplicated columns and we check for the total amount of missing values per column

```
df1.duplicated().sum()
```

```
## np.int64(0)
```

```
df1.isna().sum()
```

```
## GAME_ID                         0
## MATCHUP                         0
## LOCATION                        0
## W                               0
## FINAL_MARGIN                    0
## SHOT_NUMBER                     0
## PERIOD                          0
## GAME_CLOCK                      0
## SHOT_CLOCK                   5567
## DRIBBLES                        0
## TOUCH_TIME                      0
## SHOT_DIST                       0
## PTS_TYPE                        0
## SHOT_RESULT                     0
## CLOSEST_DEFENDER                0
## CLOSEST_DEFENDER_PLAYER_ID      0
## CLOSE_DEF_DIST                  0
## FGM                             0
## PTS                             0
## player_name                     0
## player_id                       0
## dtype: int64
```

We see that there are lots of missing values in the SHOT_CLOCK column so we will look at some of them to see if there is any pattern to the missingness. We will look at the first 10 rows with missing SHOT_CLOCK values.

```
df1_shot_clocks = df1[df1['SHOT_CLOCK'].isna()]

df1_shot_clocks[['GAME_ID', 'SHOT_CLOCK', 'GAME_CLOCK']].head(10)
```

```
##       GAME_ID  SHOT_CLOCK GAME_CLOCK
## 2    21400899         NaN       0:00
## 24   21400845         NaN       0:04
## 54   21400768         NaN       0:01
## 76   21400742         NaN       0:01
## 129  21400611         NaN       0:02
## 134  21400611         NaN       0:03
## 141  21400600         NaN       0:03
## 145  21400600         NaN       0:01
## 184  21400478         NaN       0:13
## 204  21400423         NaN       0:01
```

After looking at some examples of rows with missing SHOT_CLOCK values, we can see that most of them have less than 24 seconds left on the game clock. Let's examine this further.

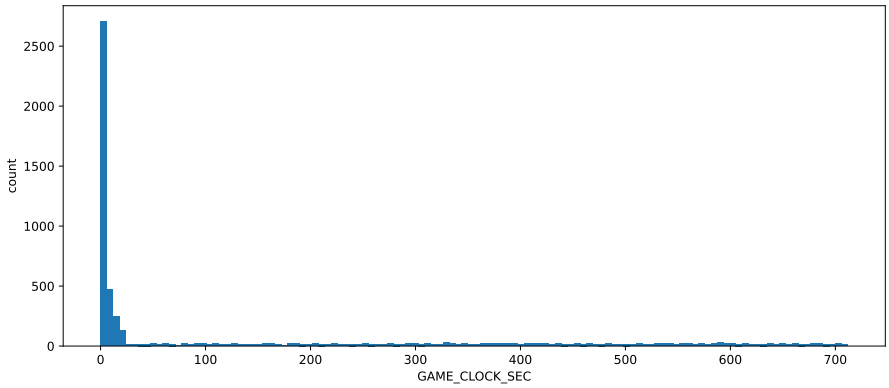


Figure 1: Distribution of NULL Shot Clock Values

It makes sense that there are more missing values of SHOT_CLOCK when there are less than 24 seconds left on the game clock, but there are still some missing values when there are more than 24 seconds left on the game clock. We will look at these cases to see if there is any pattern to the missingness.

```
null_sc_gt24 = null_sc[null_sc["GAME_CLOCK"] > 24]
null_sc_gt24['GAME_ID'].value_counts().head(10)
```

```
## GAME_ID
## 21400339    169
## 21400579    149
## 21400115    141
## 21400616    138
## 21400648    135
## 21400032    133
## 21400084    129
## 21400675    121
## 21400088    118
## 21400300     84
## Name: count, dtype: int64
```

We can see that the values missing are mostly from the same games, so it is likely that there was an issue with the data collection for those games. We will drop these rows from the dataset

```
df1 = df1.drop(null_sc_gt24.index)
```

We will replace the remaining missing values in shot clock with the game clock

```
df1["SHOT_CLOCK"] = df1["SHOT_CLOCK"].fillna(df1["GAME_CLOCK"])
```

Now, we will look at the density of each column in order to see the overall shape of each numeric variable:

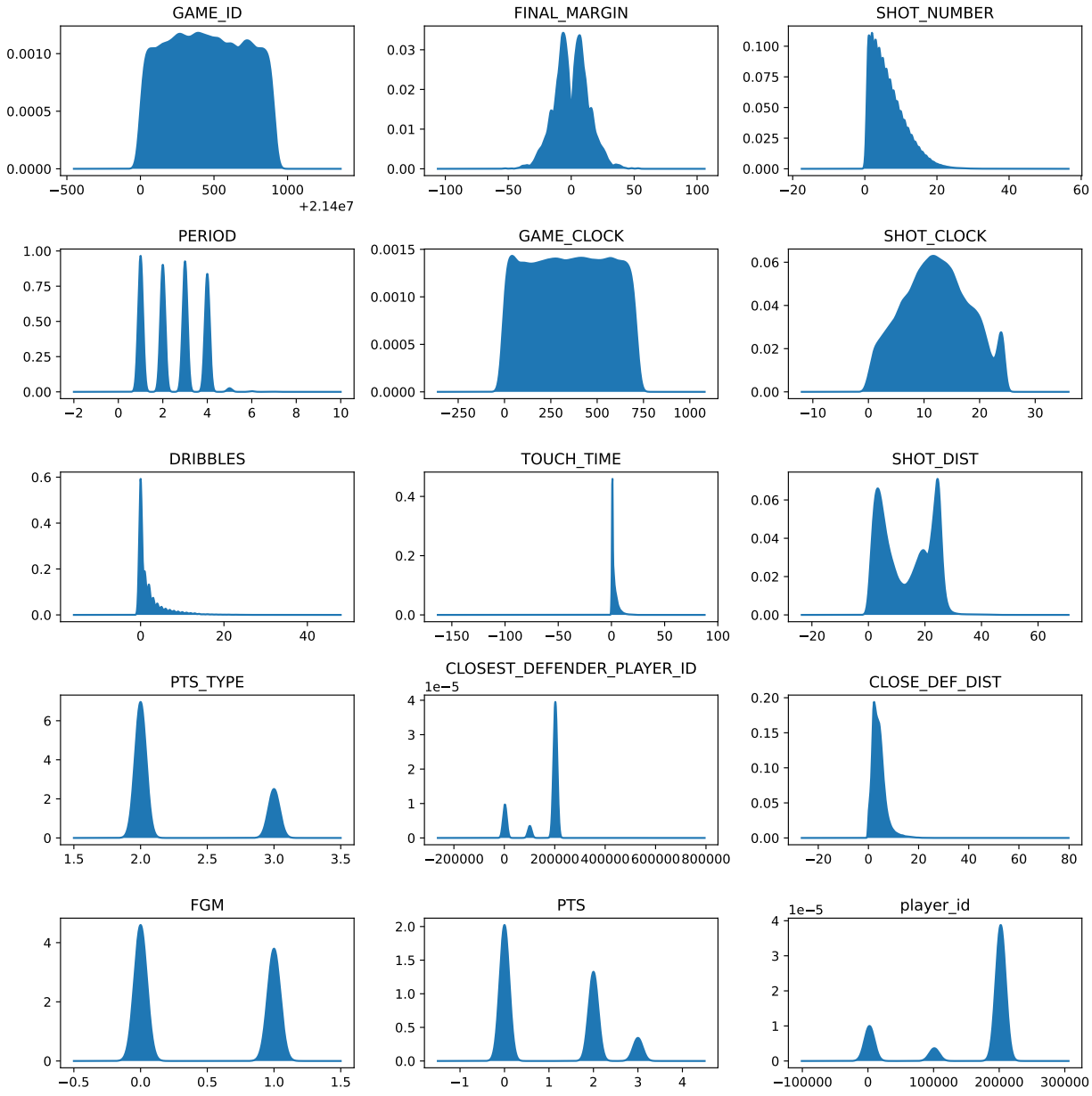


Figure 2: Distribution plots for numeric variables.

Now that we have a general idea of each covariate, we will look closely at each one and any possible interactions with other variables

Let's start by looking at shot distance. Looking at the graph, we can see 2 clear peaks: the first peak likely represents shots in the paint/close-range, whereas the second peak (around 23-25 feet) likely represents three-point shots. The valley between the peaks suggests fewer mid-range shots, which makes sense since teams tend to prefer shots either very close to the rim or beyond the 3-point line, avoiding inefficient mid-range attempts.

Building on this distribution of shot distances, it is natural to now examine how shot location interacts with defensive pressure to influence shooting efficiency.

Let's look at the relationship between shot distance, closest defender distance, and field goal percentage. Since both shot distance and defensive pressure change shot quality, a heatmap helps us see how FG% varies across these two dimensions at once

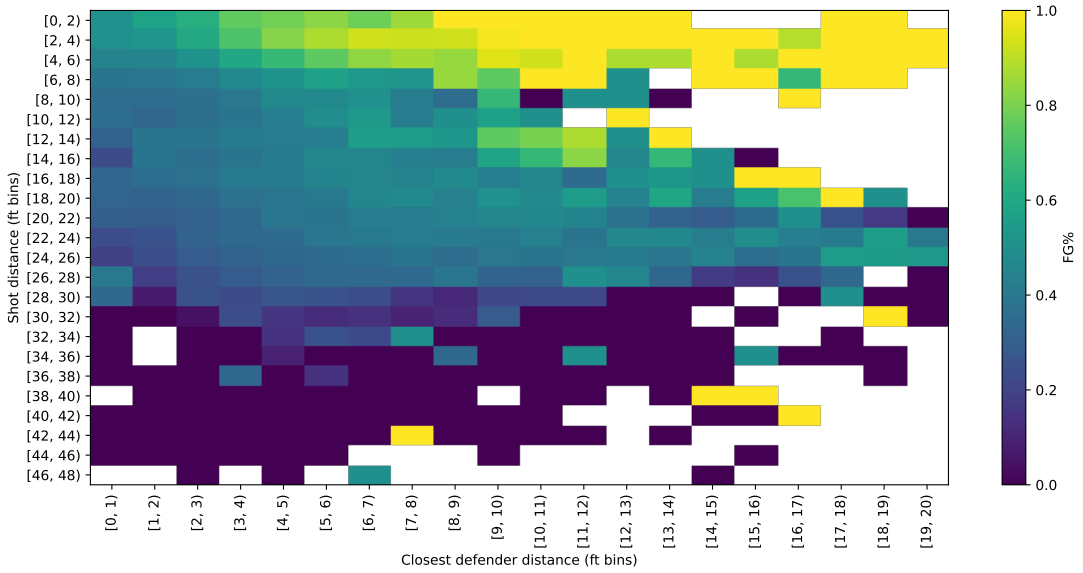


Figure 3: FG% heatmap by shot-distance bins and closest-defender-distance bins.

Looking at the heatmap, we can see that the field goal percentage is generally higher when the shot distance is shorter and when the closest defender distance is larger. This makes sense since it is easier to make a shot when you are closer to the basket and when there is more space between you and the defender. However, we can see that the shot distance has a much stronger effect on the field goal percentage than the closest defender distance, since the field goal percentage drops much more significantly as the shot distance increases than it does as the closest defender distance decreases.

Having explored how shot distance and defensive pressure affect efficiency overall, we now turn to individual defender performance to see how the most frequent defenders compare to the league average. We will create a horizontal bar chart to visualize this relationship.

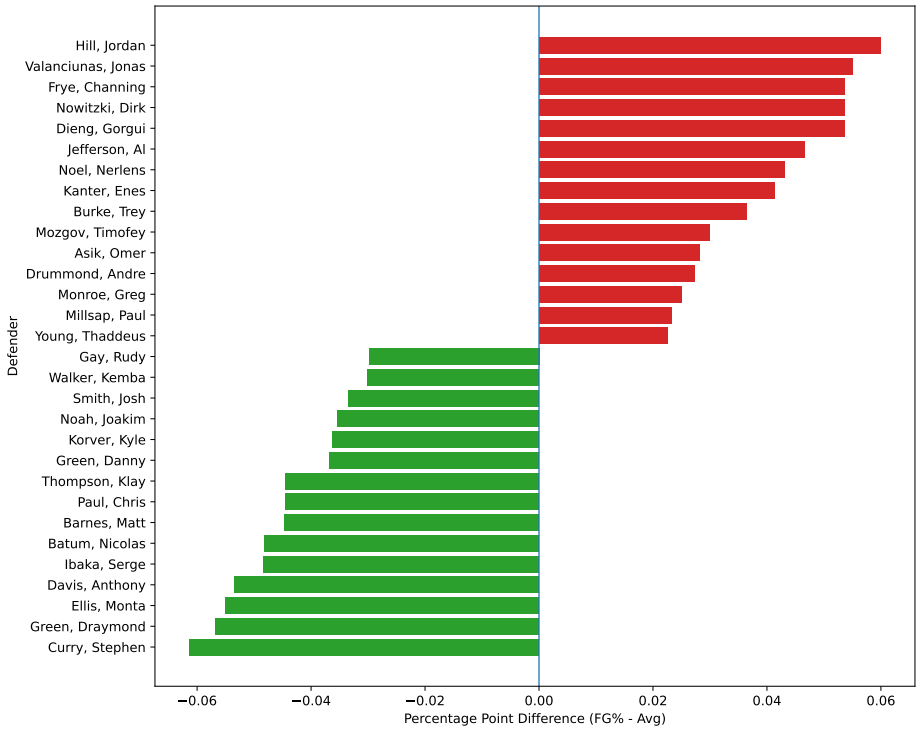


Figure 4: Defender performance: FG% difference vs league average (defenders with more than 500 attempts).

The problem with this graph is that it does not take into account things such as shot distance, the role that the defender plays in the defense (a centre may be more likely to defend shots close to the basket while a guard may be more likely to defend shots further from the basket) and other factors that can affect the field goal percentage. Therefore, we cannot conclude that the defenders with a positive difference are bad defenders and those with a negative difference are good defenders based on this graph alone.

To improve the robustness of the analysis, we incorporate shot distance into the plot, allowing for a more reliable and context-adjusted evaluation.

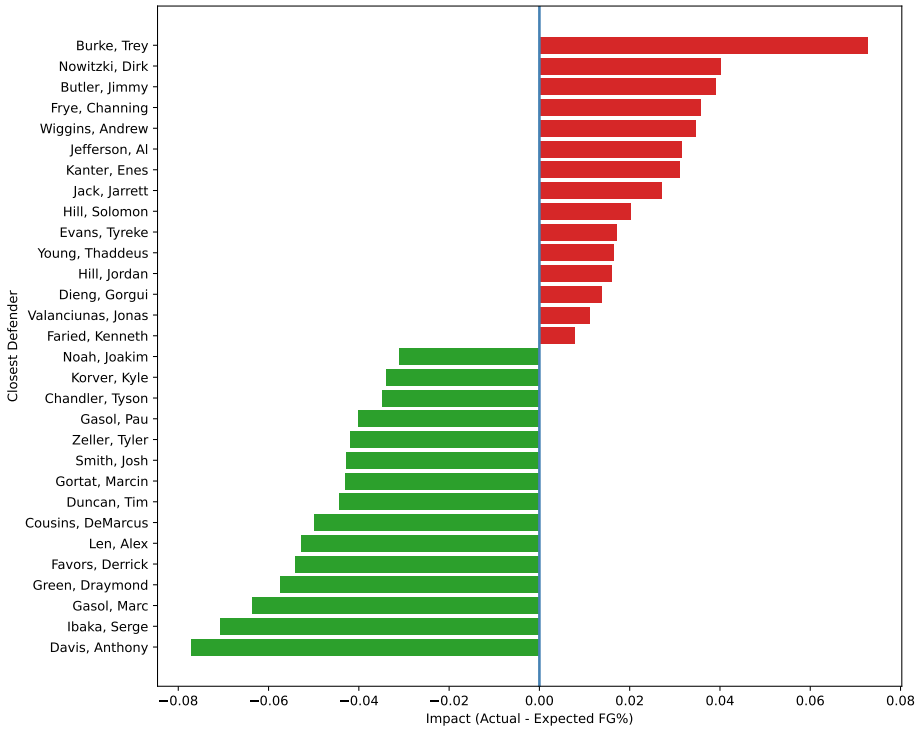


Figure 5: Adjusted defender impact (controls for shot distance).

Looking at the graph, we can see that the defender has a great impact on the shot being made or not, since we can see that good defenders such as Draymond Green (DPOY award winner) really affect the percentage of shots made. Having seen this, we can create a column in the dataset that shows the defender percentage of field goals made on them, which could be useful for the model further on.

```
d = df1.copy()

## We create shot distance bins as before
bins = np.arange(0, 52, 2)
d["SHOT_BIN"] = pd.cut(d["SHOT_DIST"], bins=bins, right=False)

# We group by bin to calculate the %FG for each bin
d["EXPECTED_FG_PCT"] = d.groupby("SHOT_BIN")["FGM"].transform("mean")

d["FG_RESID"] = d["FGM"] - d["EXPECTED_FG_PCT"]

d["DEFENDER_IMPACT_SD"] = d.groupby("CLOSEST_DEFENDER")["FG_RESID"].transform("mean")

d["CLOSEST_DEFENDER_IMPACT"] = d["DEFENDER_IMPACT_SD"]

df1 = d
```

```
## We now drop the unwanted columns created before from the dataset
df1.drop(columns=["SHOT_BIN", "EXPECTED_FG_PCT", "FG_RESID", "DEFENDER_IMPACT_SD"], inplace=True)
```

```
## We check the column was created well
df1['CLOSEST_DEFENDER_IMPACT'].head()
```

Now that we have had a look at the defender impact, we will now look at the number of dribbles and the impact that this has. The distribution of dribbles is highly right-skewed, with most shots occurring after very few dribbles, indicating that quick decision-making and ball movement dominate shot creation, while high-dribble isolation attempts are relatively rare

Since most shots are taken after very few dribbles, it's worth testing whether shot efficiency changes as dribble count increases—because more dribbles usually mean tougher, more self-created shots (often under pressure), which may lower FG%

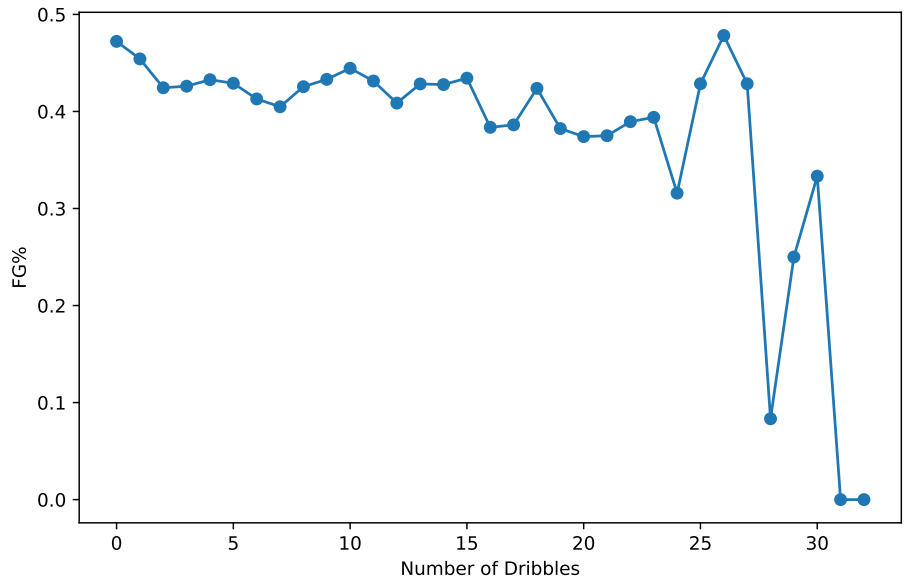


Figure 6: FG% vs Number of Dribbles.

We can see that catch and shoot shots (0 dribbles) have the highest field goal percentage, and as the number of dribbles increases, the field goal percentage generally decreases. This makes sense since catch and shoot shots are often taken when the shooter is open and has a good look at the basket, while shots taken after multiple dribbles may be more contested or taken under less ideal conditions.

Now, looking at the shotclock column, we will look at how the shot clock affects the field goal percentage. Looking at the density plot of the shot clock, we can see that there are a lot of shots taken with less than 10 seconds left on the shot clock, and the number of attempts is generally higher when there are more seconds left on the shot clock.

We also see a very big spike in the field goal attempts when there are 24 seconds left on the shot clock, which is likely due to the fact that these shots are often rebounds or putbacks after an offensive rebound.

We will look at how this relates to FG%

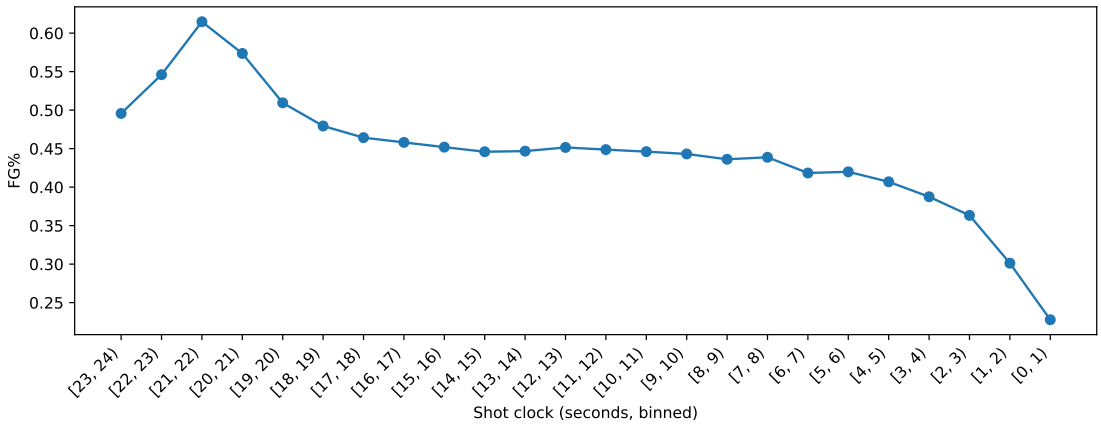


Figure 7: Overall field goal percentage by 1-second shot-clock bins (x-axis inverted: less time remaining to the right).

The graph shows that shooting efficiency is highest early in the shot clock and declines steadily as time decreases, with a sharp drop in the final seconds. This suggests that early offense produces higher-quality shots, while late-clock attempts tend to be more difficult and contested.

After observing that overall FG% declines as the shot clock runs down, it is important to separate two-point and three-point shots, since they differ in difficulty, value, and typical shot creation process. By splitting the data, we can determine whether the late-clock efficiency drop affects both shot types equally or disproportionately impacts one of them.

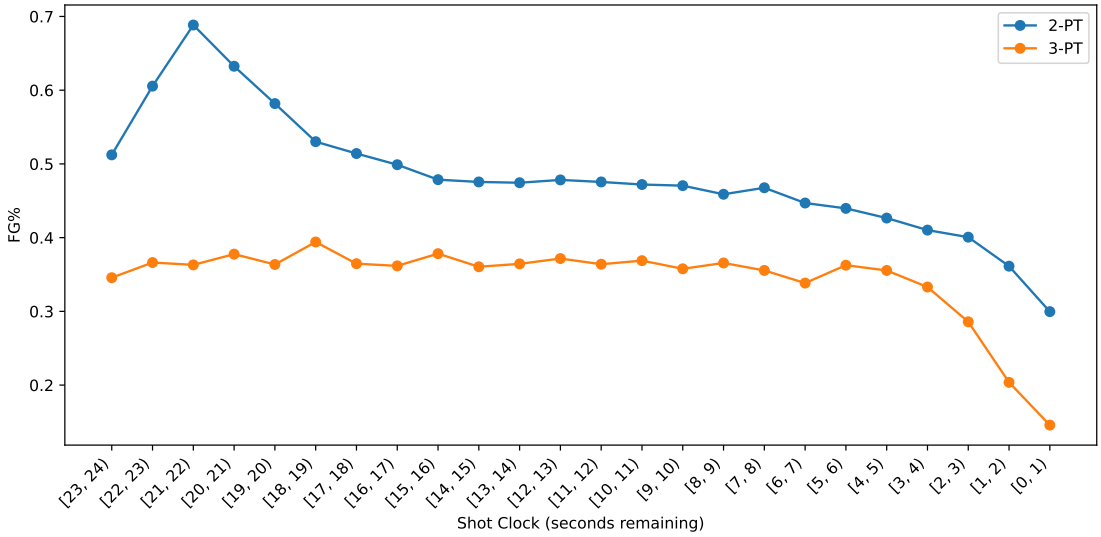


Figure 8: Field goal percentage by shot-clock bin for 2PT vs 3PT attempts.

While both shot types become less efficient as time decreases, two-point shots show a larger early-clock efficiency spike, whereas three-point efficiency remains relatively stable until the final seconds. The sharp late-clock decline for both confirms that shot quality deteriorates under time pressure.

After analysing shot clock pressure, it is natural to consider game pressure, which is often higher when the score is close. To capture this, we will look at FINAL_MARGIN to see how competitive the game was.

Although it is measured at the end of the game rather than at the time of each shot, for the EDA we will assume that games ending in a blowout were generally one-sided throughout, while games with a small final margin were competitive for most of the game.

The density of FINAL_MARGIN is concentrated around small point differences, indicating that most games are relatively competitive, while extreme blowouts are comparatively rare.

It can be interesting to investigate whether teams adjust their shot selection depending on how close the game is. In particular, three-point shots are higher variance and can quickly change the score, so examining 3PT attempt rate across different margin bins helps us understand whether teams rely more heavily on three-point shooting in certain game contexts (when trailing or in blowouts)

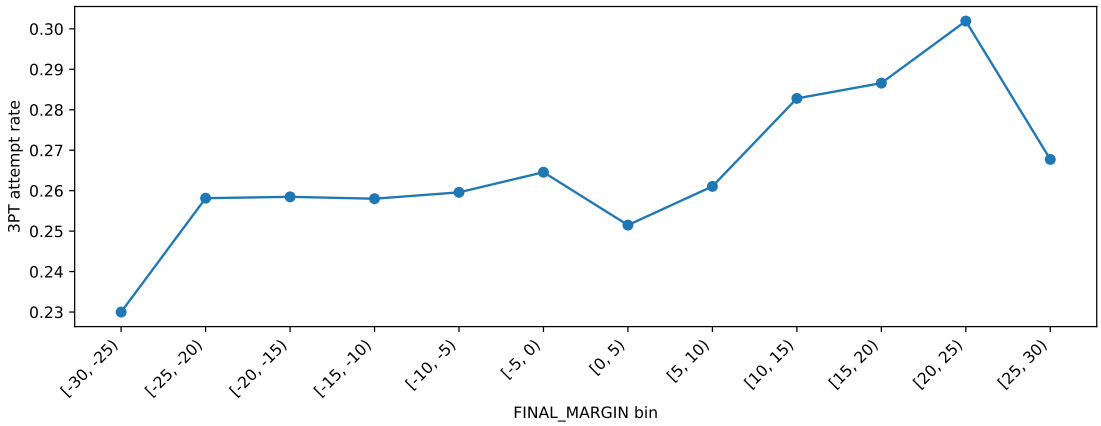


Figure 9: 3PT attempt rate across final-margin bins.

This pattern is noteworthy because it indicates that in games where a team is winning by a large margin, the three-point attempt rate is higher, which may reflect a greater willingness to take lower-percentage or higher-variance shots when the outcome is less uncertain. In tightly contested games, the three-point attempt rate is lower, suggesting a preference for more controlled and potentially higher-percentage scoring options.

Interestingly, when a team is losing by a large margin, the three-point attempt rate does not increase as much as might be expected and instead declines. This could reflect more conservative shot selection, structural changes in defense to avoid the losing team making a come back, or it could be because teams that are losing by a lot may have less skilled shooters on the court who are less likely to attempt 3PT shots.

Let's now look at how FG% differs depending on the final margin for 2pt and 3pt shots.

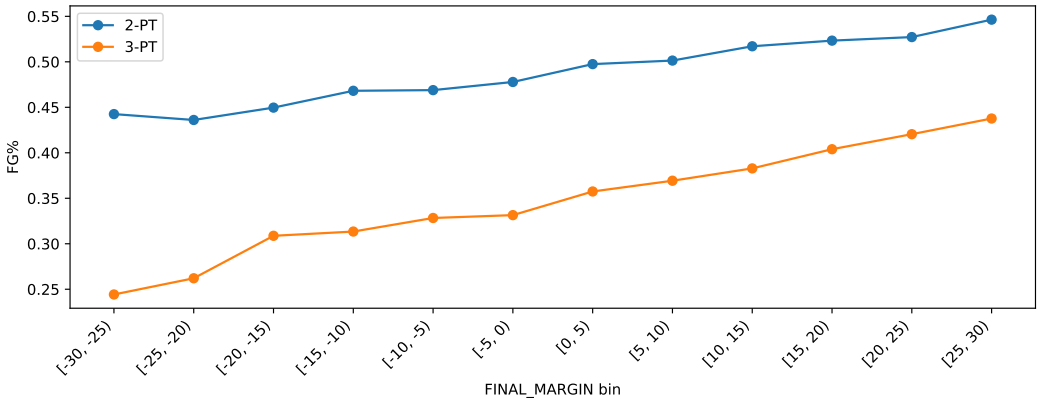


Figure 10: Field Goal percentage against final margins (2 points vs 3 points)

This graph is not very useful since the final margin does not tell us the point difference at the time of the shot, so we cannot draw any conclusions from this graph since it is obvious that the teams that win by a lot will have a higher field goal percentage than the teams that lose by a lot, and this is not necessarily because of the point difference at the time of the shot, but rather because of the overall quality of the teams and players on the court.

We will now analyse the impact of shot number. The distribution of shot number is heavily right-skewed, indicating that most players take relatively few shots per game, while a small number of high-usage players account for a disproportionately large number of attempts

Let's look at how FG percentage varies with shot number, in order to see if the hot-hand effect is true

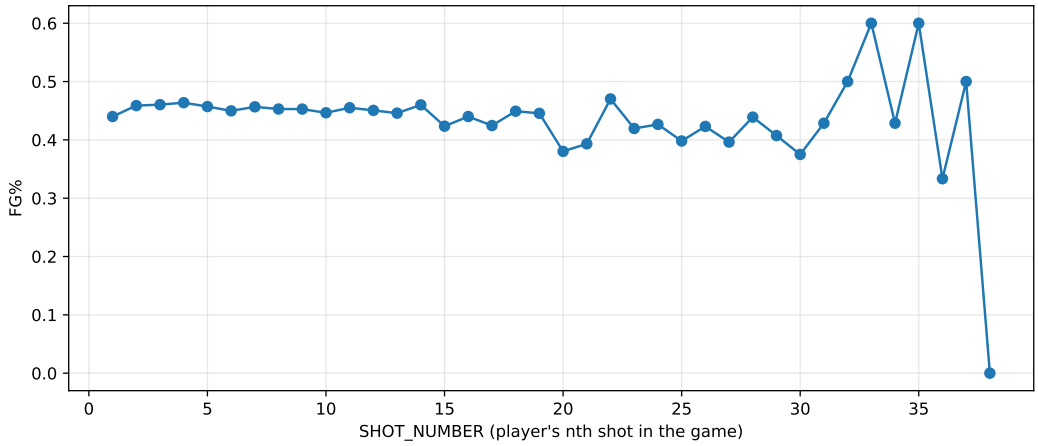


Figure 11: Field Goal percentage against Shot Number

Field goal percentage remains remarkably stable across a player's first 20 shots, providing little evidence of a systematic hot-hand effect.

To further understand player shot behaviour, we examine the relationship between shot number and shot distance. By analyzing the average shot distance for each shot number, we can investigate whether shot selection evolves over the course of a player's attempts in a game, whether players begin with higher-percentage shots and gradually take more difficult attempts, or whether usage patterns remain consistent

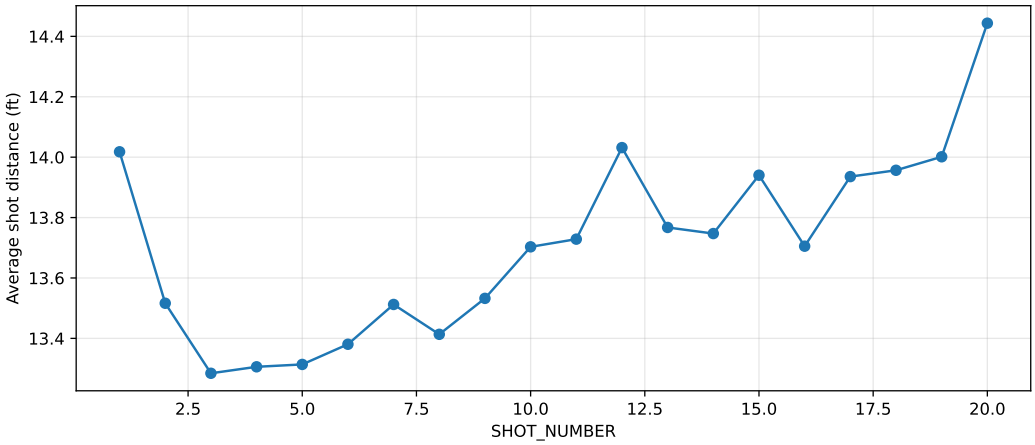


Figure 12: Shot Distance against number of shots (filtered to attempts at least 500)

The graph suggests a modest upward trend in shot distance as shot number increases, indicating that players with higher shot volumes may gradually take slightly longer attempts over the course of a game.

Having analysed shot characteristics and game context, it is also important to consider environmental factors, such as whether the team is playing at home or away.

Home-court advantage is a well-documented phenomenon in sports, potentially driven by crowd support, familiarity with the arena, travel fatigue, and referee bias. These factors may influence player confidence, shot selection, and ultimately field goal percentage.

Therefore, examining FG% by home versus away status allows us to determine whether there is a measurable efficiency difference attributable to game location, beyond the tactical and situational variables already explored

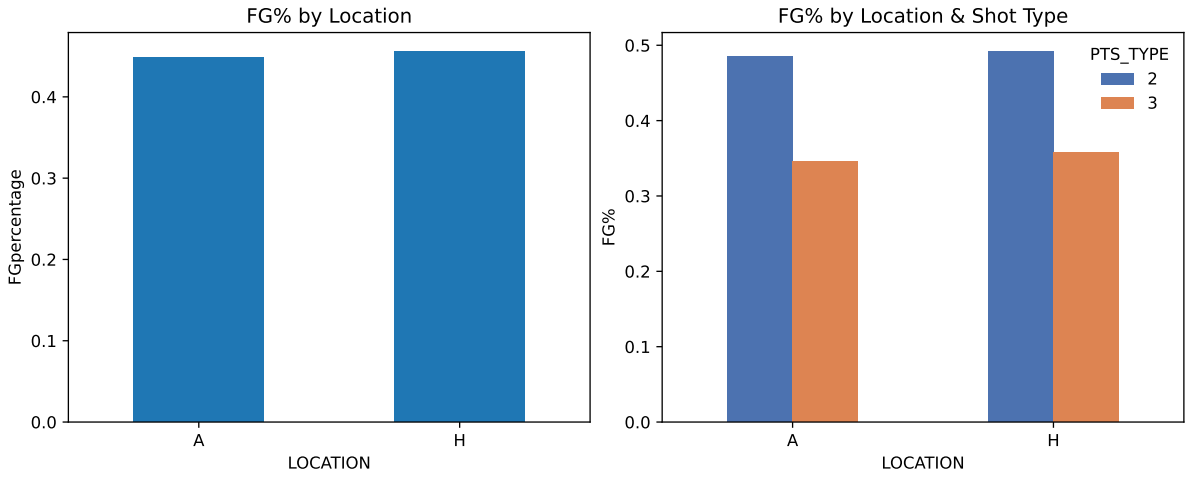


Figure 13: FG% by location and by shot type

We can see that it remains more or less the same, with a slightly higher field goal percentage at home than away, which is consistent with the well-known “home court advantage” phenomenon in sports. The difference is not very large, but it is consistent with the idea that players may perform slightly better when playing at home due to factors such as crowd support, familiarity with the court, and reduced travel fatigue.

This difference could be bigger for players in the 'clutch' moments of the game, such as in the last few minutes of a close game, where the pressure is higher and the home crowd can have a bigger impact on the players' performance. However, since we do not have the point difference at the time of the shot, we cannot analyze this aspect further with the current dataset.

Having explored individual relationships visually, the next step is to formally assess correlation between variables. While graphs allow us to identify potential trends and patterns, correlation analysis provides a quantitative measure of the strength and direction of linear relationships.

By computing correlation coefficients, we can determine which variables are most strongly associated with each other. This helps identify key predictors, detect potential multicollinearity issues, and guide subsequent modelling decisions. Let's show this graphically:

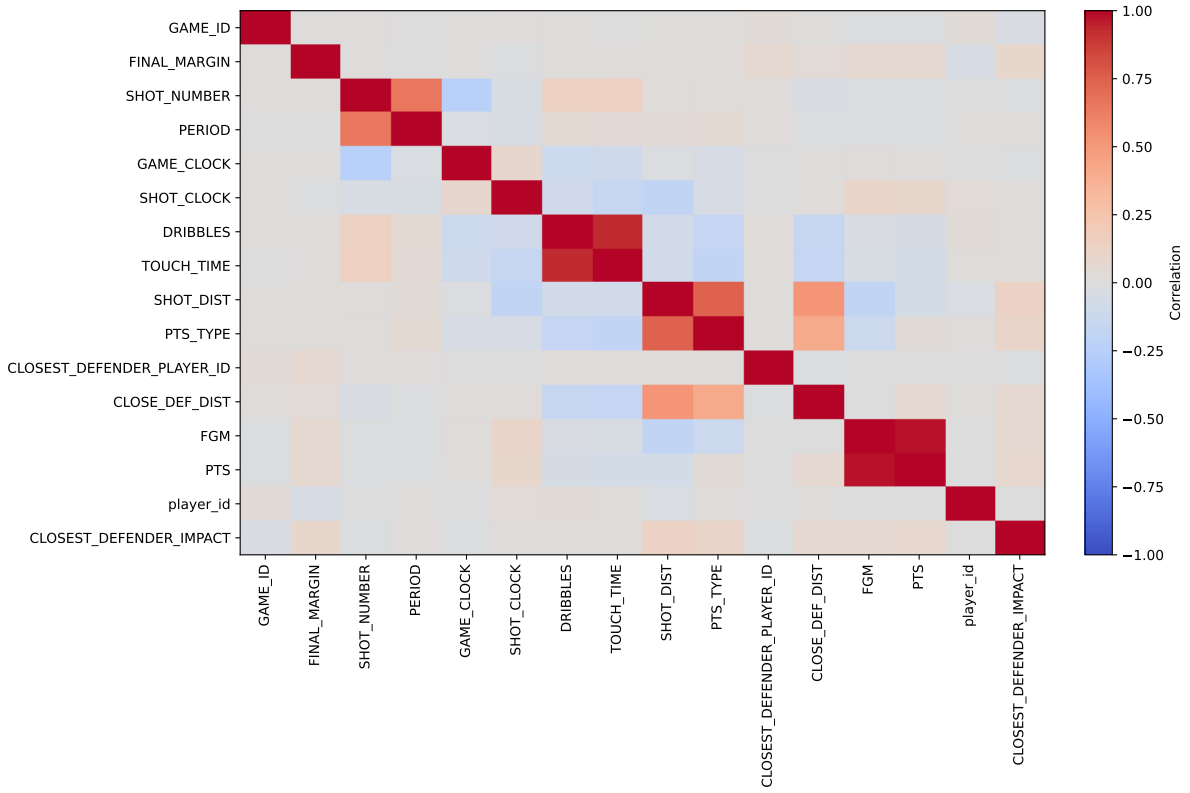


Figure 14: Correlation matrix

Looking at the correlation matrix, we can see that it highlights several pairs of variables that are strongly correlated and therefore may introduce redundancy if included together in a model. In particular, FGM and PTS show an almost perfect positive correlation, meaning both capture essentially the same information about scoring outcomes. Similarly, DRIBBLES and TOUCH_TIME are highly correlated, as longer possession time naturally involves more dribbles. Additionally, SHOT_DIST and PTS_TYPE are strongly related, since three-point shots are inherently taken from longer distances. Including both variables from each of these pairs may lead to multicollinearity and reduce model interpretability. Consequently, careful feature selection is necessary to avoid redundant predictors while retaining the most informative variables.

3. Building the Model

3.1 Selecting Covariates

For the building process of this model, we will begin by selecting a covariate set that is both predictively informative and conceptually valid for modelling the probability that an NBA shot is made. The covariates used as predictors have to be: (i) Available at the moment of the shot (ii) Clear basketball interpretation linked to shot quality or context (iii) Demonstrate a clear and reproducible association with shot success in our EDA, consistent with the expected direction and not driven by outliers.

Under these principles, we will use the following covariates for Bayesian logistic regression model:

- `SHOT_DIST` was selected as the core measure of shot difficulty. The EDA indicated a pronounced and monotone decline in FG% with increasing distance, making this variable both highly informative and straightforward to interpret.
- `PTS_TYPE (2PT vs 3PT)` was included to account for the structural difference between two- and three-point attempts. This variable captures distinct baseline conversion rates and prevents distance-related effects from being conflated with the fundamentally different shot regimes.
- `CLOSE_DEF_DIST` was chosen to represent defensive contest intensity. Greater separation from the nearest defender corresponds to higher shot quality, and the EDA suggested a clear relationship with shooting success.
- `SHOT_CLOCK` was included to account for time pressure and shot creation constraints. Late-clock possessions typically involve more difficult, rushed attempts, and the EDA indicated behaviour consistent with this mechanism.
- `DRIBBLES` was retained to distinguish self-created shots from catch-and-shoot opportunities. This variable provides an interpretable link to shot balance, rhythm, and defensive attention, and showed meaningful variation in make rates across levels.
- `SHOT_NUMBER` was incorporated to capture within-game sequencing effects, reflecting changes in usage, fatigue, and selection as games progress. While more indirect than the variables above, it provides a defensible "game context" control and was therefore treated as a candidate covariate in the main model.
- `LOCATION (home vs away)` was added as a parsimonious contextual control. It is available at shot time, easy to interpret, and helps absorb systematic differences related to venue effects.

Two variables were excluded on conceptual grounds despite any apparent association in the EDA:

- `FINAL_MARGIN` was removed because it is a post-game outcome rather than information available at the time of the shot. Using it would introduce future information relative to the shot attempt, undermining interpretability and yielding a model that is not valid for decision-time prediction.
- `CLOSEST_DEFENDER_IMPACT` was not included in the primary model because it was derived using observed shot outcomes (FGM) from the dataset. Incorporating such a feature risks target leakage and can artificially inflate apparent performance. For the main specification, defender effects are therefore captured directly via `CLOSE_DEF_DIST`, which is outcome-independent.

This selection procedure yields a covariate set that reflects the principal determinants of shot success identified in the EDA, supporting both interpretable inference and informative prediction in the Bayesian framework.

3.2 Choosing a Prior

Prior specification is an extensive area of study, and when it comes to logistic regression there are many choices available. The choice of a prior is one of the most important steps when building a model, as it can have large impacts on the resulting posterior inferences. For this reason, the prior choice will not be made arbitrarily, but it will be based on the framework proposed by Gelman et al., which provides a general approach for prior specification for logistic regression. (Gelman et al. (2008))

Let

$$\beta = \left(\beta_0, \beta_1, \ \ldots \ , \beta_p\right)^\top$$

denote the coefficient vector in our model. The predictors included in the model are shot distance (`SHOT_DIST`), closest defender distance (`CLOSE_DEF_DIST`), shot clock (`SHOT_CLOCK`), number of dribbles (`DRIBBLES`), shot type (`PTS_TYPE`), shot number (`SHOT_NUMBER`), and location (`LOCATION`).

One of the main challenges when specifying a prior is getting the scale right. Since the numerical predictors are all measured on different scales, specifying parameters such as prior variance becomes difficult to interpret consistently across coefficients. Therefore, Gelman et al. proposes to first centre and scale all predictors, to make prior specification easily comparable across coefficients. In particular, it is suggested to scale all numerical predictors to have mean 0 and standard deviation 0.5. Hence, following this framework, we will start by standardising our numerical predictors (`SHOT_DIST`, `CLOSE_DEF_DIST`, `SHOT_CLOCK`, `DRIBBLES`, and `SHOT_NUMBER`) to the scale proposed by Gelman et al., and retaining our binary predictors (`PTS_TYPE` and `LOCATION`) as indicators for interpretability.

Now that we have appropriate scales for our predictors, we can begin with the specification of the prior. Gelman et al. suggest 3 different prior distributions and reports that the best performing one for logistic regression is the Cauchy prior (which is a student-t distribution with 1 degree of freedom). In particular, for a weakly informative prior, Gelman et al. suggests a Cauchy distribution centred at 0 with scale parameter 2.5 (when the data has been standardized), which we will use as a baseline when specifying our priors.

Firstly, we start by specifying the priors for which we don't necessarily expect directional bias, i.e. our initial assumption is that the coefficients will be centred around zero (no effect on the response). These are `SHOT_CLOCK`, `DRIBBLES`, `SHOT_NUMBER`, and `LOCATION`, as there is no strong indication that these will have a positive or negative effect on the data. Hence the prior for these variables will be:

$$\beta_j \overset{\text{ind}}{\sim} \text{Cauchy}(0, 2.5),$$
$$j = (\texttt{SHOT\_CLOCK, DRIBBLES, SHOT\_NUMBER, LOCATION})$$

For the rest of the variables, where we expect a clearer direction of effect, we will be shifting the location parameter slightly away from zero to reflect our expectations.

Firstly, for `SHOT_DIST`, we expect a negative effect on the probability of making a shot, i.e. the further away from the basket the less likely it is for the shot to go in. Therefore, we will shift the location parameter by -0.5, to give:

$$\beta_{\texttt{SHOT\_DIST}} \sim \text{Cauchy}(-0.5, 2.5)$$

Give the scaling we used, this means that before observing the data, we expect a 2 standard deviation increase in `SHOT_DIST` to lead to a decrease of the odds of making a shot by a factor of $e^{-0.5}$, which is approximately 0.61.

Furthermore, for the variable `CLOSE_DEF_DIST`, we expect a positive effect on the response variable, i.e. the further away the closest defender the easier it is to make a shot. We will shift the location parameter by 0.5 to give:

$$\beta_{\texttt{CLOSE\_DEF\_DIST}} \sim \text{Cauchy}(0.5, 2.5)$$

Since `CLOSE_DEF_DIST` was also scaled according to Gelman et al.'s framework, this means that before observing the data, we expect a 2 standard deviation increase in `CLOSE_DEF_DIST` to lead to an increase in the odds making a shot by a factor of $e^{0.5}$, which is approximately 1.61.

For the variable `PTS_TYPE`, we expect a negative effect on the response variable. The variable is coded such that $0 =$ `2_POINT_SHOT` and $1 =$ `3_POINT_SHOT`, hence, we expect 3 point shots to be harder to make than 2 point shots. We will therefore shift the location parameter by 0.25 to give:

$$\beta_{\texttt{PTS\_TYPE}} \sim \text{Cauchy}(-0.25, 2.5)$$

It is important to note that the scaling for our binary variables is different to that of numeric variables so interpretation of coefficients differs slightly. Here, this means that before observing the data, we expect that changing the shot from 2 point to 3 point leads to a decrease in the odds by a factor of $e^{-0.25}$, which is approximately 0.78.

Finally, for the intercept parameter β_0, Gelman et al. recommmends specifying a Cauchy prior centred at 0, but with a higher scale parameter of 10, i.e.:

$$\beta_0 \sim \text{Cauchy}(0, 10)$$

Now that we have chosen all of our priors, we can build the regression.

3.3 Coding the Model

3.3.1 Setting up Covariates

We will now turn categorical variables into binary (0/1) variables so they can be used cleanly in our logistic regression model. Logistic regression needs numerical inputs, so we recode them into indicator variables: PTS_TYPE = 1 means a three-point attempt (and 0 means a two-point attempt), while LOCATION = 1 means the shot was taken at home (and 0 means away).

```
df1["PTS_TYPE"] = np.where(df1["PTS_TYPE"] == 3, 1, 0)

df1["LOCATION"] = np.where(df1["LOCATION"] == 'H', 1, 0)
```

We now only keep the variables that we want for modelling

```
df1_r = py$df1

## We keep only the variables that we want

model_df1 <- df1_r[, c("FGM",
                       "SHOT_DIST",
                       "CLOSE_DEF_DIST",
                       "SHOT_CLOCK",
                       "DRIBBLES",
                       "PTS_TYPE",
                       "SHOT_NUMBER",
                       "LOCATION")]

y <- model_df1$FGM
```

3.3.2 Creating the k-fold assignments

We start off by creating the k-folds

```
### 1. Create folds (k-fold CV structure)

make_folds <- function(n, K = 5, seed = 1) {
  set.seed(seed)
  sample(rep(1:K, length.out = n))
}
K <- 5
fold_id <- make_folds(nrow(df1_r), K = K, seed = 123)
table(fold_id)
```

```
## fold_id
##     1     2     3     4     5
## 25212 25211 25211 25211 25211
```

3.3.3 Scaling Covariates

Now, we apply the scaling proposed by Gelman et. al. using the formula:

$$x^* = \frac{x - \bar{x}}{2\, sd(x)}$$

```
### 2. Gelman scaling (training fold only; no data leakage)

# We save the continuous variables to scale
cont_vars <- c("SHOT_DIST", "CLOSE_DEF_DIST", "SHOT_CLOCK", "DRIBBLES", "SHOT_NUMBER")

# We compute scaling constants on the training fold
get_scaling_constants <- function(train_df, cont_vars) {
  info <- list()
  for (v in cont_vars) {
    mu <- mean(train_df[[v]])
    s  <- sd(train_df[[v]])
    if (s == 0) stop(paste("Zero SD in training fold for variable:", v))
    info[[v]] <- list(mean = mu, sd = s)
  }
  info
}

# We apply Gelman scaling using those constants
apply_gelman_scaling <- function(df_part, cont_vars, info) {
  out <- df_part
  for (v in cont_vars) {
    mu <- info[[v]]$mean
    s  <- info[[v]]$sd
    out[[paste0(v, "_sc")]] <- (out[[v]] - mu) / (2 * s)
  }
  out
}
```

3.3.4 Design Matrix and Priors

We now build the design matrix X and response y using the scaled versions of continuous variables. The design matrix X is what will go into:

$$\eta = X\beta p = \sigma(\eta)$$

where σ is the sigmoid (logistic) function derived earlier:

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

```
### 3. Build the design matrix X and response y

build_X <- function(df_scaled) {
  X <- cbind(
    Intercept      = 1,
    SHOT_DIST      = df_scaled$SHOT_DIST_sc,
    CLOSE_DEF_DIST = df_scaled$CLOSE_DEF_DIST_sc,
    SHOT_CLOCK     = df_scaled$SHOT_CLOCK_sc,
    DRIBBLES       = df_scaled$DRIBBLES_sc,
    PTS_TYPE       = df_scaled$PTS_TYPE,
    SHOT_NUMBER    = df_scaled$SHOT_NUMBER_sc,
    LOCATION       = df_scaled$LOCATION
  )
  X
}
```

```
### 4. Prior vectors (location + scale by coef)

prior_loc <- c(
  Intercept      = 0,
  SHOT_DIST      = -0.5,
  CLOSE_DEF_DIST = 0.5,
  SHOT_CLOCK     = 0,
  DRIBBLES       = 0,
  PTS_TYPE       = -0.25,
  SHOT_NUMBER    = 0,
  LOCATION       = 0
)


prior_scale <- c(
  Intercept      = 10,
  SHOT_DIST      = 2.5,
  CLOSE_DEF_DIST = 2.5,
  SHOT_CLOCK     = 2.5,
  DRIBBLES       = 2.5,
  PTS_TYPE       = 2.5,
  SHOT_NUMBER    = 2.5,
  LOCATION       = 2.5
)
```

3.3.5 Building train/validation datasets per fold

```
### 5. We now create one fold's train/val set

make_fold_data <- function(df, fold_id, k, cont_vars) {
  train_df <- df[fold_id != k, , drop = FALSE]
  val_df   <- df[fold_id == k, , drop = FALSE]

  # We compute scaling constants on train only
  info <- get_scaling_constants(train_df, cont_vars)

  # We apply scaling to train and val using train constants
  train_sc <- apply_gelman_scaling(train_df, cont_vars, info)
  val_sc   <- apply_gelman_scaling(val_df,   cont_vars, info)

  # We build the design matrices
  X_tr <- build_X(train_sc)
  y_tr <- train_sc$FGM

  X_va <- build_X(val_sc)
  y_va <- val_sc$FGM

  list(
    X_tr = X_tr, y_tr = y_tr,
    X_va = X_va, y_va = y_va,
    scaling_info = info
  )
}

# We create the folds

fold_results <- vector("list", K)

for (k in 1:K) {

  fd <- make_fold_data(df1_r, fold_id, k = k, cont_vars = cont_vars)

  fold_results[[k]] <- fd
}

lapply(1:K, function(k) dim(fold_results[[k]]$X_tr))
```

```
## [[1]]
## [1] 100844      8
##
## [[2]]
## [1] 100845      8
##
## [[3]]
## [1] 100845      8
##
## [[4]]
## [1] 100845      8
```

```
##
## [[5]]
## [1] 100845      8
```

```
lapply(1:K, function(k) dim(fold_results[[k]]$X_va))
```

```
## [[1]]
## [1] 25212     8
##
## [[2]]
## [1] 25211     8
##
## [[3]]
## [1] 25211     8
##
## [[4]]
## [1] 25211     8
##
## [[5]]
## [1] 25211     8
```

3.3.6 The Posterior Distribution

Now, we write the posterior using the likelihood and the prior. Since in logistic regression the likelihood involves a product of many probabilities, for larger n these becomes extremely small. Therefore, by taking the logarithm, products are converted into sums and ratios into differences which improves numerical stability. Our posterior distribution is given by:

$$p(\beta \mid y, X) \propto \left(\prod_{i=1}^{n} p_i^{y_i} \cdot (1 - p_i)^{1-y_i} \right) \cdot p(y), \quad p_i = \frac{1}{1 + e^{-x_i^\top \beta}}$$

So the log posterior is given by:

$$\log p(y \mid X, \beta) = \sum_{i=1}^{n} [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Simplifying this using logarithm properties:

$$\log(\sigma(\eta)) = \eta - \log(1 + e^{\eta}), \qquad \log(1 - \sigma(\eta)) = -\log(1 + e^{\eta}),$$

we obtain:

$$\log p(y \mid X, \beta) = \sum_{i=1}^{n} [y_i \eta_i - \log(1 + e^{\eta_i})]$$

We will use this to code since it is more computationally efficient.

Having specified the likelihood, we now introduce prior distributions for the regression coefficients.

We assume independent Cauchy priors on the coefficients:

$$\beta_j \sim \text{Cauchy}(m_j, s_j), \qquad j = 0, 1, ..., p - 1,$$

where m_j is the location (prior centre) and $s_j > 0$ is the scale. In our model, some m_j are chosen to be nonzero in order to reflect directional beliefs. The scale is the same for all priors (2.5) except for the intercept, which has scale 10.

The Cauchy density is:

$$p(\beta_j) = \frac{1}{\pi s_j \left[1 + \left(\frac{\beta_j - m_j}{s_j}\right)^2\right]}.$$

Therefore the log-prior for a single coefficient is:

$$\log p(\beta_j) = -\log(\pi s_j) - \log\left(1 + \left(\frac{\beta_j - m_j}{s_j}\right)^2\right),$$

and under independence, the joint log-prior becomes:

$$\log p(\beta) = \sum_{j=0}^{p-1} \log p(\beta_j).$$

Bayes' theorem gives the posterior distribution:

$$p(\beta \mid y, X) \propto p(y \mid X, \beta)\, p(\beta).$$

Taking logs gives the log-posterior (up to an additive constant):

$$\log p(\beta \mid y, X) = \log p(y \mid X, \beta) + \log p(\beta) + C,$$

where C does not depend on β. Since Metropolis–Hastings only requires ratios of posterior densities, this constant cancels and does not need to be computed.

Substituting the expressions above, the quantity evaluated inside the MH acceptance ratio is:

$$\underbrace{\sum_{i=1}^{n} [y_i \eta_i - \log(1 + e^{\eta_i})]}_{\text{log-likelihood}} + \underbrace{\sum_{j=0}^{p-1} \left[-\log(\pi s_j) - \log\left(1 + \left(\frac{\beta_j - m_j}{s_j}\right)^2\right)\right]}_{\text{log-prior}}.$$

This log-posterior defines the target distribution for the MH algorithm: the Markov chain is constructed so that its stationary distribution is $p(\beta \mid y, X)$.

```r
### 6. Log-likelihood, log-prior, log-posterior

## Log Likelihood
loglik <- function(beta, X, y) {
  eta <- as.vector(X %*% beta)
  sum(y * eta - log(1 + exp(eta)))
}

# Log-prior
logprior <- function(beta, prior_loc, prior_scale, X = NULL) {
  beta <- as.numeric(beta)
```

```r
  if (!is.null(X)) {
    nm <- colnames(X)
    prior_loc   <- prior_loc[nm]
    prior_scale <- prior_scale[nm]
  }

  m <- as.numeric(prior_loc)
  s <- as.numeric(prior_scale)


  sum(-log(pi * s) - log1p(((beta - m) / s)^2))
}

#Log-posterior
logpost <- function(beta, X, y, prior_loc, prior_scale) {
  loglik(beta, X, y) + logprior(beta, prior_loc, prior_scale, X = X)
}
```

3.3.7 The Proposal Distribution

Once we have coded all of this, we can start coding the algorithm.

We employ a Gaussian random-walk Metropolis proposal for the regression coefficients, proposing new parameter values according to

$$\beta' = \beta^{(t)} + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \Sigma).$$

This proposal is symmetric, meaning the Metropolis-Hastings acceptance probability depends only on the difference in log-posterior values. To select the proposal covariance matrix Σ, we follow the optimal scaling results for random-walk Metropolis algorithms, (Roberts, Gelman, and Gilks (1997)) Specifically, we set

$$\Sigma = \frac{2.38^2}{d}\,\widehat{\mathrm{Var}}(\hat{\beta}),$$

where d is the number of parameters and $\widehat{\mathrm{Var}}(\hat{\beta})$ is the estimated covariance matrix obtained from the standard logistic regression fit.

3.3.8 Acceptance step

After we generate a proposed parameter vector β' from the random-walk proposal, we need to decide whether to move the chain to this new point or stay where we are. Metropolis-Hastings does this by comparing how likely the proposed value is under the posterior.

Because our proposal is symmetric ($\beta' = \beta^{(t)} + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, \Sigma)$), the proposal densities cancel, and the acceptance probability depends only on the posterior:

$$\alpha = \min\left(1,\ \frac{p(\beta' \mid y, X)}{p(\beta^{(t)} \mid y, X)}\right).$$

We compute this using the log-posterior:

$$\log \alpha = \log p(\beta' \mid y, X) \ - \ \log p(\beta^{(t)} \mid y, X).$$

Then we draw $u \sim \text{Uniform}(0,1)$. - If $\log(u) < \log \alpha$, we accept and set $\beta^{(t+1)} = \beta'$. - Otherwise we reject and keep the current value: $\beta^{(t+1)} = \beta^{(t)}$.

This accept/reject step makes the chain spend more time in high-posterior regions while still allowing occasional moves away, which is necessary to explore the full posterior distribution.

```
### 7. Create function for 1 MH step
mh_step <- function(beta_curr, lp_curr, X, y, prior_loc, prior_scale, U) {
  p <- length(beta_curr)

  # Propose step
  z <- rnorm(p)
  eps <- as.vector(t(U) %*% z)
  beta_prop <- beta_curr + eps

  # Accept/reject step
  lp_prop <- logpost(beta_prop, X, y, prior_loc, prior_scale)
  log_alpha <- lp_prop - lp_curr

  if (log(runif(1)) < log_alpha) {
    list(beta = beta_prop, lp = lp_prop, accepted = TRUE)
  } else {
    list(beta = beta_curr, lp = lp_curr, accepted = FALSE)
  }
}
```

3.3.9 Running the Algorithm

Once we have defined a single Metropolis-Hastings update, we obtain a full Markov chain by repeating this step many times. At iteration t, the current state $\beta^{(t-1)}$ is updated to $\beta^{(t)}$. We store each draw $\beta^{(t)}$ in a matrix, with one row per iteration and one column per coefficient. We also store whether each proposal was accepted in a logical vector. Repeating this procedure generates a sequence of dependent draws whose stationary distribution is the posterior $p(\beta \mid y, X)$. After running for a sufficiently large number of iterations, we can discard an initial burn-in period and use the remaining draws to approximate posterior summaries.

The first part of the Markov chain can be influenced by the initial values, so we discard an initial set of iterations called the burn-in period. After removing burn-in, the remaining draws are treated as samples from the stationary distribution, which is the posterior $p(\beta \mid y, X)$. Using these post burn-in draws, we summarize each coefficient by computing posterior quantities such as the posterior mean and standard deviation, and credible intervals.

After obtaining posterior draws of β, we can generate predictions for the validation fold. For each posterior draw $\beta^{(s)}$, we compute the linear predictor $\eta^{(s)} = X_{\text{val}}\beta^{(s)}$ and transform it into a probability using the logistic (sigmoid) function

$$p^{(s)} = \sigma(\eta^{(s)}) = \frac{1}{1 + e^{-\eta^{(s)}}}.$$

This produces a predicted probability of a shot being made for each observation in the validation set. Because we obtain many posterior draws, each observation has several predicted probabilities. We summarise these by taking the posterior mean probability, which is the average of $p^{(s)}$ across all post burn-in draws.

To convert these probabilities into a class prediction, we apply a 0.5 threshold: if the predicted probability is greater than or equal to 0.5, the model predicts that the shot is made, otherwise it predicts a miss.

Finally, we evaluate predictive performance by comparing the predicted classes to the observed outcomes y_{val}. The proportion of correctly classified observations gives the validation accuracy, which measures how well the model predicts unseen data in the cross-validation framework.

```
### 8. Bayesian logistic regression (MH) with 5-fold CV

n_iter  <- 20000
burn_in <- 5000
acc_folds <- numeric(K)
accept_rates <- numeric(K)

beta_chain_demo <- NULL
X_demo <- NULL
y_demo <- NULL

for (k in 1:K) {

  fd <- fold_results[[k]]

  # Train/val step
  X <- fd$X_tr
  y <- fd$y_tr
  X_va <- fd$X_va
  y_va <- fd$y_va

  p <- ncol(X)

  mle_fit   <- glm(y ~ X - 1, family = binomial(link = "logit"))
  beta_curr <- as.vector(coef(mle_fit))

  Sigma_hat  <- vcov(mle_fit)
  Sigma_prop <- (2.38^2 / p) * Sigma_hat
  U <- chol(Sigma_prop)

  lp_curr <- logpost(beta_curr, X, y, prior_loc, prior_scale)

  # We run the MH chain
  beta_chain <- matrix(NA, nrow = n_iter, ncol = p)
  colnames(beta_chain) <- colnames(X)
  accept <- logical(n_iter)

  for (t in 1:n_iter) {
    out <- mh_step(beta_curr, lp_curr, X, y, prior_loc, prior_scale, U)
    beta_curr <- out$beta
    lp_curr   <- out$lp
    accept[t] <- out$accepted
    beta_chain[t, ] <- beta_curr
  }

  accept_rates[k] <- mean(accept)

  # We save one fold (fold 1) for the plots after
  if (k == 1) {
    beta_chain_demo <- beta_chain
```

```
    X_demo <- X
    y_demo <- y
  }
  # We calculate the post burn-in draws
  post_draws <- beta_chain[(burn_in + 1):n_iter, , drop = FALSE]


  eta_mat <- X_va %*% t(post_draws)
  P_mat <- 1 / (1 + exp(-eta_mat))
  p_hat <- rowMeans(P_mat)

  y_pred <- ifelse(p_hat >= 0.5, 1, 0)
  acc_folds[k] <- mean(y_pred == y_va)

  cat("Fold", k,
      "| acc =", round(acc_folds[k], 4),
      "| accept =", round(accept_rates[k], 4), "\n")
}
```

```
## Fold 1 | acc = 0.614 | accept = 0.268
## Fold 2 | acc = 0.6108 | accept = 0.273
## Fold 3 | acc = 0.6101 | accept = 0.2662
## Fold 4 | acc = 0.6051 | accept = 0.2762
## Fold 5 | acc = 0.6039 | accept = 0.2683
```

```
acc_folds
```

```
## [1] 0.6139933 0.6108048 0.6100512 0.6050930 0.6039427
```

```
mean(acc_folds)
```

```
## [1] 0.608777
```

```
sd(acc_folds)
```

```
## [1] 0.004179942
```

```
accept_rates
```

```
## [1] 0.26800 0.27300 0.26615 0.27625 0.26835
```

```
mean(accept_rates)
```

```
## [1] 0.27035
```

3.3.10 Posterior summaries

We now compute the summaries on the fold 1 “demo chain” so that we can present a single representative set of posterior estimates and uncertainty intervals, while still using all folds for the cross-validation accuracy results

```r
### 9. Posterior summaries

post_draws_demo <- beta_chain_demo[(burn_in + 1):n_iter, , drop = FALSE]

post_mean <- colMeans(post_draws_demo)
post_sd   <- apply(post_draws_demo, 2, sd)
post_ci   <- apply(post_draws_demo, 2, quantile, probs = c(0.025, 0.975))

summary_table <- data.frame(
  coef = colnames(post_draws_demo),
  mean = as.numeric(post_mean),
  sd   = as.numeric(post_sd),
  ci_2.5  = as.numeric(post_ci[1, ]),
  ci_97.5 = as.numeric(post_ci[2, ])
)

summary_table
```

```
##             coef        mean         sd       ci_2.5     ci_97.5
## 1      Intercept -0.24470653 0.01038361 -0.265655222 -0.22367330
## 2      SHOT_DIST -1.12456247 0.02167841 -1.165823691 -1.08228321
## 3 CLOSE_DEF_DIST  0.55844468 0.01763439  0.522959055  0.59237291
## 4     SHOT_CLOCK  0.22190424 0.01305683  0.195805086  0.24744599
## 5       DRIBBLES -0.12417079 0.01338950 -0.150435026 -0.09832699
## 6       PTS_TYPE  0.08795140 0.02207996  0.043521882  0.13092077
## 7    SHOT_NUMBER  0.01717352 0.01249710 -0.007911385  0.04101986
## 8       LOCATION  0.03234766 0.01281177  0.006342250  0.05853502
```

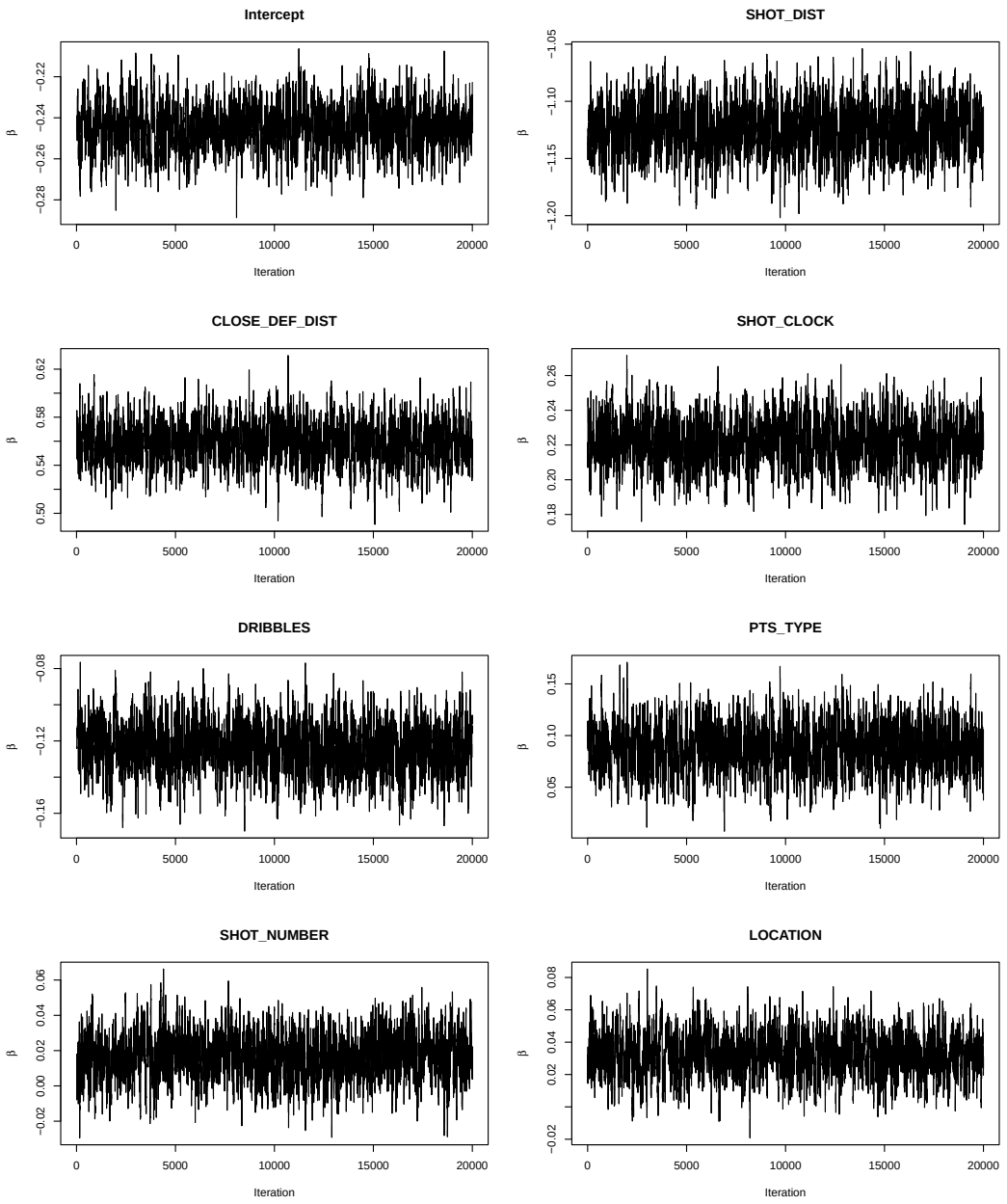
Looking at the table, all of the results were as expected. However, some covariates seem to have very little impact, such as location, which was not surprising seeing that in the EDA the difference was very small.

3.3.11 Trace Plots

```r
### 10. Trace plots for MCMC chain (on fold 1)

par(mfrow = c(4,2))

for (j in 1:ncol(beta_chain_demo)) {
  plot(beta_chain_demo[, j],
       type = "l",
       main = colnames(beta_chain_demo)[j],
       xlab = "Iteration",
       ylab = expression(beta))
}
```



These trace plots show the sampled values of each regression coefficient β across the MCMC iterations. For all parameters, the chains are around a stable horizontal band with no obvious upward or downward trend, which suggests the sampler has reached a stationary distribution after burn-in and is not drifting. The traces also show frequent movement rather than long flat sections, indicating reasonable mixing and that the algorithm is exploring the posterior rather than getting stuck. The centres of the bands are consistent with the posterior summary table: SHOT_DIST remains clearly negative, CLOSE_DEF_DIST and SHOT_CLOCK are positive, DRIBBLES is negative, and the smaller effects such as SHOT_NUMBER and LOCATION are close to zero. Overall, the plots provide visual evidence that the Metropolis–Hastings chain has converged and that the posterior summaries are based on a well-behaved sample.

3.3.12 Model Comparison (Standard Logistic Regression)

We will now compare our model with a normal standard logistic regression to see if it performs better:

```
### 11. Baseline: standard logistic regression (CV)

acc_glm_folds <- numeric(K)

for (k in 1:K) {

  fd <- fold_results[[k]]


  X_tr <- fd$X_tr
  y_tr <- fd$y_tr


  X_va <- fd$X_va
  y_va <- fd$y_va

  # We fit standard logistic regression on training fold
  glm_fit <- glm(y_tr ~ X_tr - 1, family = binomial(link = "logit"))

  # We predict probabilities on validation fold
  p_hat_glm <- as.vector(1 / (1 + exp(-(X_va %*% coef(glm_fit)))))


  y_pred_glm <- ifelse(p_hat_glm >= 0.5, 1, 0)


  acc_glm_folds[k] <- mean(y_pred_glm == y_va)

  cat("GLM fold", k, "acc =", acc_glm_folds[k], "\n")
}
```

```
## GLM fold 1 acc = 0.614033
## GLM fold 2 acc = 0.6108445
## GLM fold 3 acc = 0.6100908
## GLM fold 4 acc = 0.60545
## GLM fold 5 acc = 0.6038237
```

```
acc_glm_folds
```

```
## [1] 0.6140330 0.6108445 0.6100908 0.6054500 0.6038237
```

```
mean(acc_glm_folds)
```

```
## [1] 0.6088484
```

```
sd(acc_glm_folds)
```

```
## [1] 0.004159461
```

```r
### 12. Frequentist logistic regression summaries (on fold 1)

# We fit logistic regression on the same fold-1 training data
glm_fit_demo <- glm(y_demo ~ X_demo - 1, family = binomial(link = "logit"))


glm_mean <- coef(glm_fit_demo)
glm_sd   <- sqrt(diag(vcov(glm_fit_demo)))


glm_ci_low  <- glm_mean - 1.96 * glm_sd
glm_ci_high <- glm_mean + 1.96 * glm_sd


summary_table_glm <- data.frame(
  coef    = names(glm_mean),
  mean    = as.numeric(glm_mean),
  sd      = as.numeric(glm_sd),
  ci_2.5  = as.numeric(glm_ci_low),
  ci_97.5 = as.numeric(glm_ci_high)
)

summary_table_glm
```

```
##                   coef        mean         sd       ci_2.5     ci_97.5
## 1      X_demoIntercept -0.24449316 0.01093466 -0.265925091 -0.22306123
## 2      X_demoSHOT_DIST -1.12416298 0.02210463 -1.167488050 -1.08083791
## 3 X_demoCLOSE_DEF_DIST  0.55871687 0.01690781  0.525577566  0.59185617
## 4     X_demoSHOT_CLOCK  0.22142562 0.01356508  0.194838070  0.24801318
## 5       X_demoDRIBBLES -0.12393473 0.01375591 -0.150896317 -0.09697314
## 6       X_demoPTS_TYPE  0.08775916 0.02270722  0.043253012  0.13226530
## 7    X_demoSHOT_NUMBER  0.01799389 0.01320758 -0.007892971  0.04388075
## 8       X_demoLOCATION  0.03176845 0.01301833  0.006252524  0.05728437
```

The frequentist summary is again very close to the Bayesian one, not only in terms of the coefficient estimates but also in terms of the uncertainty around them. The standard deviations in both tables are nearly identical, which means that both approaches are assigning a very similar level of precision to the estimated effects. In most cases, the Bayesian posterior standard deviations are marginally smaller than the frequentist standard errors, although the difference is extremely slight. The only noticeable exception is `CLOSE_DEF_DIST`, where the Bayesian standard deviation is just a little larger. Overall, these differences are too small to change the interpretation, and this is reflected in the confidence and credible intervals, which are almost the same width across the two methods. A relevant point is that for variables with weaker effects, such as `SHOT_NUMBER`, the standard deviation remains large relative to the coefficient itself in both frameworks, so the interval still includes zero. This reinforces the idea that the Bayesian model is not telling a different story from the classical logistic regression, but rather confirming it while expressing uncertainty in posterior terms.

4. Conclusion

The results indicate that contextual variables available at the moment of a shot contain meaningful but limited predictive information about NBA shot outcomes. Our model achieves an average predictive accuracy of approximately 0.609, suggesting that while there are certain characteristics of a shot attempt that can influence the probability of making it, a large part of the outcome remains uncertain.

Among the variables considered, shot distance and defensive pressure seem to be the most influential determinants of shot success. Attempts taken from greater distances and those that are more closely contested are consistently associated with lower field goal percentages. This makes sense since greater distance increases technical difficulty, and tighter defensive pressure can disrupt a player's shooting form and reduce their time before release.

The low predictive accuracy highlights an important characteristic of basketball shot outcomes: even when relevant contextual variables are taken into account, a substantial component of shot success remains random. Professional players are capable of making difficult attempts and missing relatively easy ones, meaning that the outcome of any single shot has a significant stochastic element. Therefore, models based only on observable contextual features can only partially predict shooting performance.

Furthermore, the Bayesian and classical logistic regression models produce very similar predictive performance, which is not surprising given the size of the dataset. When the sample size is large, the information contained in the data tends to dominate the influence of the prior distribution in a Bayesian framework. As a result, the posterior estimates of the coefficients converge towards the maximum likelihood estimates obtained from classical logistic regression. In our case, both approaches effectively rely on the same underlying information contained in the data, leading to nearly identical parameter estimates and very similar predictive accuracy.

One limitation of the model is the choice of prior distributions in the Bayesian model. Although relatively weak priors were used to avoid making strong assumptions about the parameters, the choice of prior is still a modelling decision that can influence the results. Different priors could lead to slightly different estimates for the coefficients, especially for variables with weaker effects. While the large sample size means that the data has a stronger influence on the results, the choice of prior still represents an assumption that may affect the estimates to some extent.

Future research could improve predictive performance by adding more variables that capture more of the complexity of NBA shot attempts. In particular, player-specific effects could be included to account for differences in shooting ability between players, as well as more information describing the exact position of the shot on the court. Defensive context could also be modelled more precisely using player tracking data, which would allow the distance, angle, and movement of defenders to be measured more accurately. Additionally, game context variables such as score differential at the moment of the shot, or fatigue effects could provide further insight into shot difficulty. Including these types of features would likely allow future models to explain a bigger part of the variability in shot outcomes and improve predictive accuracy. Exploring different types of models could also help to improve predictive accuracy. In particular, more flexible machine learning methods such as random forests or neural networks may be able to capture complex, non-linear relationships between the predictors and shot outcomes that logistic regression cannot easily represent.

5. Works Cited

Becker, Dan. 2016. "NBA Shot Logs." Kaggle. https://www.kaggle.com/datasets/dansbecker/nba-shot-logs/data.

Cao, Z. et al. 2025. "What Influences Field Goal Attempts of Professional Players?" *arXiv*. https://arxiv.org/html/2503.02137v1.

Cheema, Ahmad et al. 2021. "Modeling Shot Probability in the NBA." https://www.causeweb.org/usproc/sites/default/files/usclap/2021-1/Modeling%20Shot%20Probability%20in%20the%20NBA.pdf.

Edmond, Jason. 2017. "How Can an NBA Player Be Clutch?" In *MWSUG Proceedings*. https://www.mwsug.org/proceedings/2017/AA/MWSUG-2017-AA13.pdf.

Gelman, Andrew, Aleks Jakulin, Maria Grazia Pittau, and Yu-Sung Su. 2008. "A Weakly Informative Default Prior Distribution for Logistic and Other Regression Models." https://sites.stat.columbia.edu/gelman/research/unpublished/priors11.pdf.

Gordovil-Merino, A. et al. 2010. "Bayesian Logistic Regression in Clinical Data Analysis." *Journal of Clinical Epidemiology*. https://pubmed.ncbi.nlm.nih.gov/20524554/.

Mila, A. L., and T. J. Michailides. 2008. "Bayesian Modeling of Disease Prediction." *Plant Disease*. https://pubmed.ncbi.nlm.nih.gov/18943503/.

National Basketball Association. 2026. "NBA Stats." https://www.nba.com/stats.

Roberts, Gareth O., Andrew Gelman, and Walter R. Gilks. 1997. "Weak Convergence and Optimal Scaling of Random Walk Metropolis Algorithms." *Annals of Applied Probability*. https://projecteuclid.org/journals/annals-of-applied-probability/volume-7/issue-1/Weak-convergence-and-optimal-scaling-of-random-walk-Metropolis-algorithms/10.1214/aoap/1034625254.full.

Terhanian, George. 2015. "Advice on Making the Most of Basketball Three-Point Shot Data." https://thesportjournal.org/article/advice-on-making-the-most-of-basketball-three-point-shot-data.